eBay-style Proxy Bidding with Hidden Reserve Price 60

EDSL shared-state experiment setup

October 10, 2026

Abstract

This standalone document specifies the shared-state implementation and planned execution for this archived llm-auction condition. It is a setup paper: no new inference has been run for this condition, so it reports no model outcomes.

1 Condition and protocol

eBay-style Proxy Bidding with Hidden Reserve Price 60. This condition uses 3 bidders, private values, and 100 configured auction repetitions with model gpt-4 at temperature 0.5. Each auction has 10 periods, with bidders acting serially within each period. An interview reads the current public price and the acting bidder's own maximum before its action is committed.

Item	Setup
Condition	ebay_reserve_60
Bidders	3
Value model	private
Configured repetitions	100
Model	gpt-4
Temperature	0.5
Inference status	Not run; this file is a setup paper
Machine	Defined; diagram generated from EDSL Machine.show_diagram()

bottomrule

Table 1: Experiment setup; no inference results are included.

2 Shared-state machine

Each repetition has its own scope in the shared-state map. The machine definition is $M = (C, X, A, V, \delta, \kappa)$: constants C, internal state X, action interface A, public view V, action transition δ, and close transition κ. Separate scopes ensure that a transition in auction r cannot alter auction r'. The state contains maxima m_i, private values v_i, public actions, period t, and current price p. A submitted amount a_i updates $m_i \leftarrow \max(m_i, a_i)$. With increment Δ, the price is $\min(m_{(1)}, m_{(2)} + \Delta)$ when the second maximum is positive, and the start price otherwise. At close, the top maximum wins if the final price meets the reserve.

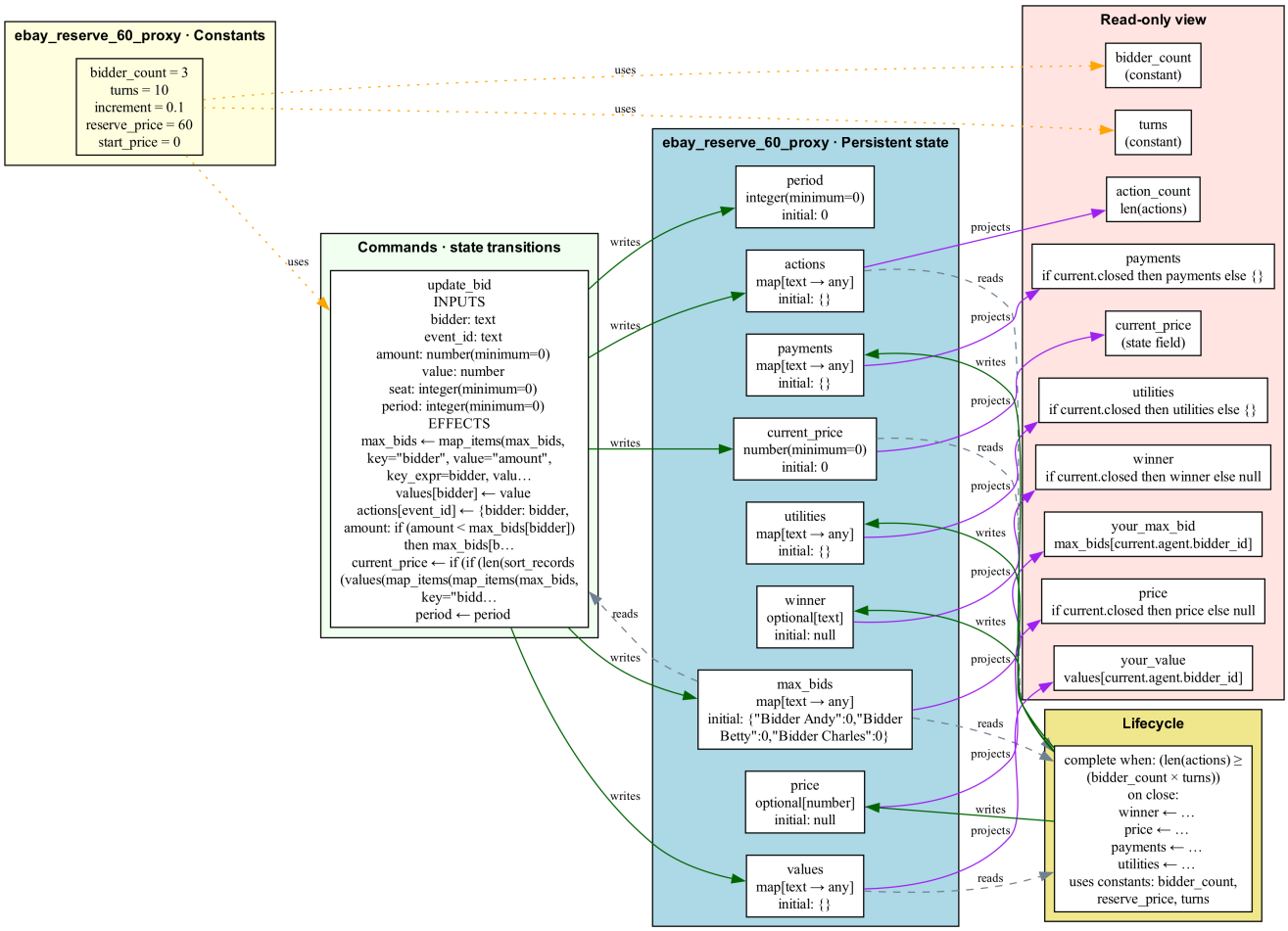


Figure 1: Diagram generated by EDSL's `Machine.show_diagram()`.

2.1 Python machine factory

The following is the implementation used to build this condition's machine.

```python
def build_proxy_machine(config: dict[str, Any]) -> Machine:
    """Build an eBay/Amazon proxy-bid machine with serial public price updates."""
    auction, rule, value_cfg = config["auction"], config["rule"], config["value"]
    count, turns = auction["number_agents"], rule["turns"]
    names = [f"Bidder {n}" for n in NAMES[:count]]
    initial_bids = {name: 0 for name in names}
    # A candidate maximum map includes the current agent's submitted maximum.
    valid_amount = choose(
        arg("amount") < field("max_bids").get(arg("bidder")),
        field("max_bids").get(arg("bidder")), arg("amount"),
    )
    updated_bids = map_items(
        field("max_bids"), key="bidder", value="amount",
        key_expr=local("bidder"),
        value_expr=choose(local("bidder") == arg("bidder"), valid_amount, local("amount")),
    )
    ranked = reduce(
        "sort_records",
```

```
    map_items(updated_bids, key="bidder", value="amount",
              key_expr=local("bidder"),
              value_expr=record(bidder=local("bidder"), amount=local("amount"))).values(),
    fields=("amount", "bidder"), descending=(True, False),
)
top = ranked.first()
second = choose(ranked.length() > 1, ranked.at(1).get("amount"), 0)
current_price = choose(
    second > 0,
    choose(top.get("amount") < second + constant("increment"),
           top.get("amount"), second + constant("increment")),
    constant("start_price"),
)
sale = current_price >= constant("reserve_price")
winner = choose(sale, top.get("bidder"), None)
final_ranked = reduce(
    "sort_records",
    map_items(field("max_bids"), key="bidder", value="amount",
              key_expr=local("bidder"),
              value_expr=record(bidder=local("bidder"), amount=local("amount"))).values(),
    fields=("amount", "bidder"), descending=(True, False),
)
final_sale = field("current_price") >= constant("reserve_price")
final_winner = choose(final_sale, final_ranked.first().get("bidder"), None)
payments = map_items(
    field("max_bids"), key="bidder", value="amount",
    key_expr=local("bidder"),
    value_expr=choose(local("bidder") == final_winner, field("current_price"), 0),
)
utilities = map_items(
    field("max_bids"), key="bidder", value="amount",
    key_expr=local("bidder"),
    value_expr=choose(
        local("bidder") == final_winner,
        field("values").get(local("bidder")) - field("current_price"), 0,
    ),
)
# amounts may only be raised; EDSL serial order makes each update visible
# to the next bidder in a round.
return Machine(
    name=f"{config['experiment']['name']}_proxy",
    constants={"bidder_count": count, "turns": turns,
               "increment": value_cfg["increment"],
               "reserve_price": rule["reserve_price"],
               "start_price": rule.get("start_price", 0),
               "soft_close": bool(rule.get("closing", False))},
    fields={
        "max_bids": state_field(StateType.map(), initial_bids),
        "values": state_field(StateType.map(), {}),
        "actions": state_field(StateType.map(), {}),
        "period": state_field(StateType.integer(minimum=0), 0),
        "current_price": state_field(StateType.number(minimum=0), rule.get("start_price", 0)
            ),
        "winner": state_field(StateType.optional(StateType.text()), None),
        "price": state_field(StateType.optional(StateType.number()), None),
        "payments": state_field(StateType.map(), {}),
```

```
            "utilities": state_field(StateType.map(), {}),
        },
        commands={"update_bid": Command(
            inputs={"bidder": StateType.text(), "event_id": StateType.text(),
                    "amount": StateType.number(minimum=0),
                    "value": StateType.number(), "seat": StateType.integer(minimum=0),
                    "period": StateType.integer(minimum=0)},
            effects=(
                assign("max_bids", updated_bids),
                put("values", arg("bidder"), arg("value")),
                put("actions", arg("event_id"), record(
                    bidder=arg("bidder"), amount=valid_amount, period=arg("period"),
                    seat=arg("seat"))),
                assign("current_price", current_price),
                assign("period", arg("period")),
            ),
        )},
        view={
            "bidder_count": constant("bidder_count"),
            "turns": constant("turns"),
            "current_price": field("current_price"),
            "your_max_bid": field("max_bids").get(current("agent.bidder_id", default=""), 0),
            "your_value": field("values").get(current("agent.bidder_id", default=""), 0),
            "action_count": field("actions").length(),
            "winner": choose(current("closed"), field("winner"), None),
            "price": choose(current("closed"), field("price"), None),
            "payments": choose(current("closed"), field("payments"), {}),
            "utilities": choose(current("closed"), field("utilities"), {}),
        },
        complete_when=field("actions").length() >= constant("bidder_count") * constant("turns")
            ,
        close_effects=(assign("winner", final_winner),
                       assign("price", choose(final_sale, field("current_price"), None)),
                       assign("payments", payments), assign("utilities", utilities)),
    )
```

3 Launch command

The EDSL runner executes locally and uses the model service and key configured for this experiment. This setup step did not launch interviews. To run the configured number of repetitions:

```
python -m edsl_replication.shared_state_auctions \\
  configs/experiments/02\_04\_ebay\_reserve\_60.yaml \\
  --output edsl_replication/studies/ebay\_reserve\_60/raw_results.json
```

4 Results

Results are pending. After inference, the condition report will be rebuilt from the EDSL result object, interview answers, machine event history, and exit snapshots. No outcome estimates or charts are included in this setup paper.